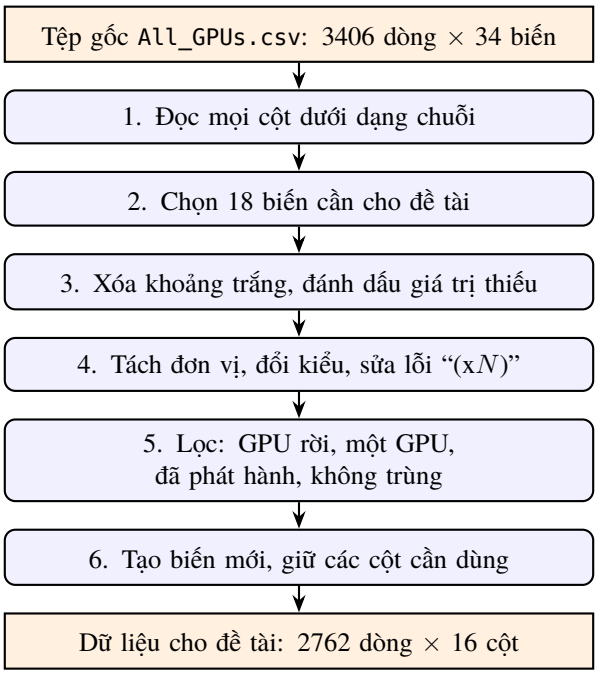
3 Tiền xử lý số liệu

Phần này trình bày quá trình chuyển tệp `All_GPUs.csv` (3406 dòng, 34 biến, mô tả ở phần tổng quan) thành bộ dữ liệu dùng cho các phân tích ở mục sau. Toàn bộ quy trình viết bằng Python (thư viện `pandas`), chạy lại được từ đầu và cho cùng một kết quả. Quy trình gồm sáu bước: đọc dữ liệu, chọn biến, làm sạch chuỗi, xử lý định dạng, lọc quan sát và tạo biến mới (Hình 1); cách xử lý dữ liệu khuyết được trình bày ở cuối mục.

Nhóm tuân theo ba nguyên tắc: không sửa tay dữ liệu gốc; mỗi hàm đọc giá trị chỉ chấp nhận đúng định dạng đã biết và dừng chương trình nếu gặp giá trị lạ; không điền giá trị cho biến mục tiêu `Max_Power`. Việc xử lý ngoại lai và biến đổi logarit thuộc về từng phân tích nên được thực hiện ở các mục sau.



Hình 1: Sơ đồ khối quy trình tiền xử lý

3.1 Đọc dữ liệu

Nhóm đọc mọi cột dưới dạng chuỗi và chưa chuyển đổi giá trị nào, để mọi giá trị đều được kiểm tra định dạng ở các bước sau. Mỗi dòng được gán mã `gpu_id` là vị trí của dòng trong tệp gốc để truy vết.

Đoạn code 1: Đọc dữ liệu thô dưới dạng chuỗi

```
raw = pd.read_csv(path, dtype="str", keep_default_na=False)
```

3.2 Chọn biến

Trong 34 biến gốc, nhóm chỉ giữ 18 biến cần cho đề tài:

- biến mục tiêu `Max_Power` và các biến giải thích `TMUs`, `Core_Speed`, `Memory`, `Memory_Bandwidth`, `Process`;

- các biến phân nhóm và định danh: `Name`, `Manufacturer`, `Notebook_GPU`, `Memory_Type`, `Release_Date`, `Architecture`;
- các biến chỉ dùng trong lúc xử lý: `Dedicated` (lọc GPU rời), `ROPs` (sửa lỗi "(xN)" ở bước xử lý định dạng), `Texture_Rate` (kiểm tra lỗi và tính hiệu quả năng lượng), `Boost_Clock`, `Memory_Bus`, `Memory_Speed` (kiểm tra lỗi).

Các biến còn lại như cổng xuất hình, độ phân giải, phiên bản DirectX, giá bán hay bộ nguồn khuyến nghị không liên quan tới câu hỏi nghiên cứu nên không được xử lý tiếp.

3.3 Làm sạch chuỗi

Các biến được chọn chứa nhiều giá trị không chuẩn: có các ký tự xuống dòng và khoảng trắng thừa (ngày phát hành ghi dạng `"\n01-Mar-2009 "`), ký hiệu `"-"` thay cho giá trị thiếu (907 giá trị của `Core_Speed`, cùng 29 giá trị chỉ chứa ký tự trắng) và 30 giá trị ghi `"Unknown Release Date"`. Nhóm xóa khoảng trắng hai đầu, rồi đổi các ký hiệu trên thành giá trị thiếu chuẩn `NA`.

Đoạn code 2: Xóa khoảng trắng và đánh dấu giá trị thiếu

```
text = selected.apply(lambda column: column.str.strip())
text = text.replace(["", "-"], pd.NA)
text["Release_Date"] = text["Release_Date"].replace(
   "Unknown Release Date", pd.NA)
```

Nhận xét. Nếu chỉ đếm dữ liệu trống như mặc định của `pandas`, tỉ lệ thiếu của `Core_Speed` và `Release_Date` sẽ bị đánh giá thấp. Vì vậy mọi tỉ lệ thiếu trong mục này đều được tính sau khi đã đổi các ký hiệu trên thành `NA`.

3.4 Xử lý định dạng

Phần lớn biến số được ghi kèm đơn vị. Nhóm tách phần số và đổi sang kiểu số theo mẫu định dạng riêng của từng biến (Bảng 1); giá trị nào không khớp mẫu thì chương trình dừng và báo lỗi.

Bảng 1: Một số định dạng gốc và kết quả chuyển đổi

Biến gốc	Giá trị gốc	Sau chuyển đổi
`Max_Power`	`141 Watts`	141 (W)
`Core_Speed`	`738 MHz`	738 (MHz)
`Memory_Bandwidth`	`64GB/sec`	64 (GB/s)
`Memory_Bandwidth`	`800MB/sec`	0,8 (GB/s)
`Process`	`55nm`	55 (nm)
`ROPs`	`16 (x2)`	32
`Release_Date`	`01-Mar-2009`	năm 2009

Nhận xét.

- 4 dòng ghi băng thông theo MB/sec được đổi sang GB/s.
- Giá trị dạng "a (xN)" (305 dòng của `ROPs`) là phép nhân: Radeon RX 580 ghi `"16 (x2)"` và có $16 \times 2 = 32$ ROPs.

- Ở đúng các dòng này, `TMUs` bị ghi **chia cho** N. Bằng chứng là tốc độ xử lý texture phải bằng TMUs nhân xung nhịp, nhưng 274 dòng không thỏa; sau khi nhân lại với N thì còn 0 dòng. Ví dụ GTX 1080 Ti được ghi 56 TMUs, giá trị đúng là 224. Lỗi này quan trọng vì `TMUs` (số đơn vị xử lý texture) phản ánh độ lớn của GPU và là một biến giải thích công suất trong mô hình hồi quy; nếu không sửa, các card đời 2016–2017 sẽ bị xem như card nhỏ hơn nhiều so với thực tế. `Memory_Bus` cũng bị chia cho N ở các dòng này và được sửa theo cùng cách.

Đoạn code 3: Sửa lỗi "(xN)": nhân lại TMUs và Memory_Bus với N

```
multiplier = parse_multiplier(selected["ROPs"]) # "16 (x2)" -> 2, "64" -> 1
tmus = parse_number(selected["TMUs"]) * multiplier
memory_bus_bit = parse_number(selected["Memory_Bus"], "Bit") * multiplier
```

3.5 Lọc quan sát

Đề tài nghiên cứu *một GPU rời đã bán ra thị trường*, nên nhóm loại GPU tích hợp, card chưa phát hành và các cấu hình nhiều GPU (ghép nhiều card SLI/CrossFire, card hai chip như GTX 690 hay R9 295X2), vì công suất ghi cho các cấu hình này là của cả hệ. Cấu hình nhiều GPU được nhận diện bằng các quy tắc trên tên sản phẩm: tên chứa "SLI", "CrossFire", "X2"… hoặc thuộc danh sách các card hai chip đã biết.

Bảng 2: Số dòng còn lại sau từng bước lọc

Bước	**Số dòng**
Dữ liệu gốc	3406
Chỉ giữ GPU rời	3018
Loại cấu hình nhiều GPU	2775
Loại card chưa phát hành	2762
Loại dòng trùng hoàn toàn	2762

Nhận xét. Bước đầu loại 374 GPU tích hợp và 14 dòng không rõ loại GPU. Dữ liệu còn 2762 GPU rời, gồm 2318 card desktop và 444 card laptop.

3.6 Tạo biến mới

Nhóm tạo thêm các biến mà nhiều phân tích phải dùng cùng một định nghĩa, rồi giữ lại 16 cột cho bộ dữ liệu cuối (Bảng 3, kèm tỉ lệ khuyết của từng cột):

- `segment`: Desktop hoặc Laptop.
- `needs_aux_power`: công suất trên 75 W, mức tối đa một khe PCIe cấp được.
- `memory_type_group`: loại bộ nhớ, gộp các loại có dưới 30 card thành "Other" để mỗi nhóm đủ lớn khi so sánh.
- `texture_rate_per_watt`: hiệu quả năng lượng (tốc độ xử lý texture trên mỗi watt).

Bảng 3: Các cột của bộ dữ liệu sau tiền xử lý và tỉ lệ khuyết

Vai trò	Cột	Khuyết (%)
Định danh	gpu_id	0,0
	name	0,0
Biến mục tiêu	max_power_w	17,4
Biến tạo từ mục tiêu	needs_aux_power	17,4
	texture_rate_per_watt	24,2
Biến giải thích	tmus	13,0
	core_speed_mhz	22,0
	memory_bandwidth_gb_s	1,3
	memory_mb	0,9
	process_nm	10,5
Biến phân nhóm	manufacturer	0,0
	segment	0,0
	memory_type	0,7
	memory_type_group	0,7
Thời gian	release_year	1,1
Kiến trúc chip	architecture	1,5

3.7 Xử lý dữ liệu khuyết

Biến mục tiêu max_power_w thiếu ở 481 dòng (17,4%). Nhóm **không điền** giá trị thiếu, vì điền bằng trung bình hay trung vị sẽ làm sai lệch phân phối; mỗi phân tích tự chọn các dòng có đủ biến cần dùng.

Các biến giải thích cũng có giá trị khuyết, nhiều nhất là core_speed_mhz (22,0%) và tmus (13,0%) (Bảng 3). Vì vậy số dòng dùng được cho một phân tích phụ thuộc vào các biến mà phân tích đó cần, và mô hình dùng nhiều biến sẽ có ít dòng hơn.

Nhận xét. Card thiếu số liệu công suất là card đời cũ hơn (trung vị năm phát hành 2011 so với 2013, kiểm định Mann–Whitney $p < 0{,}001$) và có tỉ lệ AMD cao hơn (53,1% so với 39,0%, $p < 0{,}001$). Vì vậy các thống kê trên toàn mẫu đại diện nhiều hơn cho card đời mới.

3.8 Các hàm chính và cấu trúc dữ liệu

Quy trình được tóm tắt ở Hình 1; Bảng 4 giải thích các hàm chính. Dữ liệu được lưu dưới dạng bảng pandas.DataFrame, mỗi dòng là một GPU, mỗi cột là một biến.

Kết quả. Bộ dữ liệu sau tiền xử lý gồm 2762 GPU rời và 16 cột, được dùng cho các phân tích ở các mục sau. Bảng 5 là một vài dòng của bộ dữ liệu: các biến số đã ở dạng số với một đơn vị thống nhất, giá trị thiếu được ghi là NA, và GTX 1080 Ti đã có số TMUs đúng.

Bảng 4: Các hàm chính trong quy trình tiền xử lý

Hàm	**Chức năng**
`read_raw`	Đọc tệp gốc, mọi cột dưới dạng chuỗi
`select_columns`	Giữ các biến cần cho đề tài
`clean_text`	Xóa khoảng trắng, đánh dấu giá trị thiếu
`parse_columns`	Tách đơn vị, đổi kiểu, sửa lỗi "(xN)"; báo lỗi nếu gặp định dạng lạ
`check_consistency`	Kiểm tra giá trị bằng đẳng thức giữa các biến
`filter_rows`	Lọc quan sát, đếm số dòng còn lại sau mỗi bước
`derive_variables`	Tạo biến mới, giữ các cột cần dùng

Bảng 5: Một vài dòng của bộ dữ liệu sau tiền xử lý

Tên	**Năm**	**Công suất** (W)	**TMUs**	**Xung nhịp** (MHz)	**Băng thông** (GB/s)	**Bộ nhớ** (MB)	**Tiến trình** (nm)
GeForce 8800 GT	2007	105	56	600	57,6	512	65
GeForce 9600 GS	2008	NA	24	NA	24,0	768	65
GeForce GTX 1080 Ti	2017	220	224	1480	484,4	11264	16
Radeon RX 580 8GB	2017	150	144	NA	256,0	8192	14